

# Unit 4: Covariance Matrices, PCA, and Stochastic Calculus

Charles Rambo

UCLA Anderson

2026

# Table of Contents I

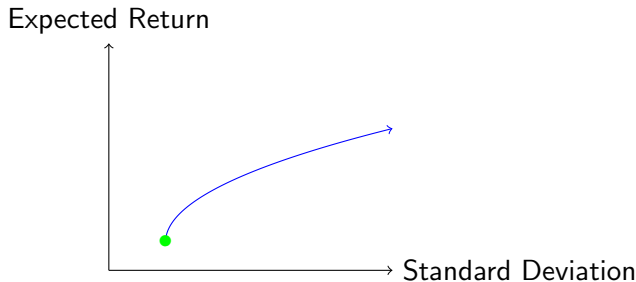
- 1 Efficient Frontier
- 2 Principal Component Analysis (PCA)
- 3 Clipping Covariance Matrices
- 4 Stochastic Calculus
  - Introduction
  - Quadratic Variation
  - Itô Integrals

# Efficient Frontier

# Efficient Frontier Definition

## Definition

The **efficient frontier** is the set of portfolios which satisfy the condition that no other portfolio in the investment universe exists with a higher expected return and the same standard deviation of return.



# Portfolio Construction Example

## Example

Consider assets 1 and 2 with monthly expected returns of 1% and 0.5% and standard deviations of 10% and 6%, respectively. Suppose the correlation between returns is 25%.

- (a) Find the minimum variance portfolio.
- (b) Graph the efficient frontier.

Assume that you cannot take short positions in this investment universe.

# Portfolio Construction Example Python Code

```
# Import modules
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar
from matplotlib.ticker import PercentFormatter

# Use LaTeX and increase resolution
plt.rcParams['text.usetex'] = True
plt.rcParams['figure.dpi'] = 300

# Use Seaborn style
plt.style.use('seaborn-v0_8')

# Define expected returns
mu1, mu2 = 0.01, 0.005

# Define standard deviations
sigma1, sigma2 = 0.10, 0.06

# Define correlation
rho = 0.25

# Create function for portfolio expected return
port_mu = lambda w: mu1 * w + mu2 * (1 - w)

# Create function for portfolio standard deviations
def port_sigma(w):
    # Calculate variance...
    # ... uncorrelated variance
    var = w**2 * sigma1**2 + (1 - w)**2 * sigma2**2
    # ... variance due to correlation
    var += 2 * w * (1 - w) * rho * sigma1 * sigma2
    # Take square root to obtain standard deviation
    return np.sqrt(var)
```

# Portfolio Construction Example Solution

```
# Get weights for minimum volatility portfolio; no short positions
# Since the square root is monotonic for positive inputs same as
# minimum variance portfolio
w_min_var = minimize_scalar(port_sigma, bounds = (0, 1), method = 'bounded').x

# Define number of samples
samples = 100

# Get weights
wt_vals = np.linspace(w_min_var, 1.0, samples)

# Calculate expected returns for each wt
mu_vals = port_mu(wt_vals)

# Calculate std of returns for each wt
sigma_vals = port_sigma(wt_vals)

# Stack results, so dimension (-1, 2)
port_results = np.column_stack([mu_vals, sigma_vals])

# Convert to data frame
port_results = pd.DataFrame(port_results, columns = ['mu', 'sigma'])
```

# Portfolio Construction Example Solution

```
# Plot efficient frontier
plt.plot(port_results['sigma'], port_results['mu'], label = 'Efficient Frontier')

# Add dot for minimum variance portfolio
plt.scatter(port_results.loc[0, 'sigma'], port_results.loc[0, 'mu'],
            label = 'Minimum Variance', color = 'green', zorder = 3)

# Add dot for maximum expected return
plt.scatter(port_results.loc[samples - 1, 'sigma'], port_results.loc[samples - 1, 'mu'],
            label = 'Maximum Expected Return', color = 'red', zorder = 3)

# Add legend
plt.legend()

# Add x-label
plt.xlabel(r'$\sigma$', fontsize = 15)

# Add y-label
plt.ylabel(r'$\mu$', fontsize = 15)

# Add title
plt.title('Efficient Frontier')

# Write both axes as a percentage
plt.gca().xaxis.set_major_formatter(PercentFormatter(1.0, decimals = 1))
plt.gca().yaxis.set_major_formatter(PercentFormatter(1.0, decimals = 1))

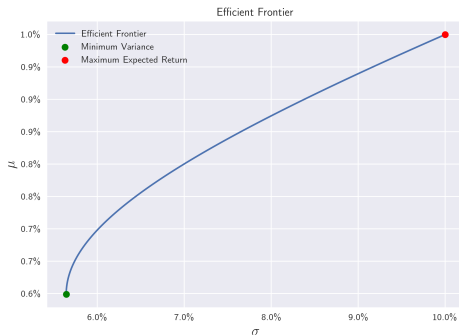
# Save the figure
plt.savefig(path + r'ex4-1.png')

plt.show()

print(r'The minimum variance portfolio has respective weights in',
      rf'assets 1 and 2 of {w_min_var:.1%} and {1 - w_min_var:.1%}.',
      f'The standard deviation of the minimum variance portfolio is {port_sigma(w_min_var)
      :.1%}.')
```

# Portfolio Construction Result

The minimum variance portfolio has respective weights in assets 1 and 2 of 19.8% and 80.2%. The standard deviation of the minimum variance portfolio is 5.6%.



# Principal Component Analysis (PCA)

# Eigenvector and Eigenvalue Ordering Convention

Suppose we have eigenvectors  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$  with corresponding eigenvalues  $\lambda_1, \lambda_2, \dots, \lambda_n$ . We will assume that

$$i < j \quad \text{implies} \quad \lambda_i \geq \lambda_j.$$

In other words, assume we have rearranged our eigenvalues and corresponding eigenvectors so that the eigenvalues are in descending order.

# Eigenvectors and Eigenvalues

From the spectral theorem, we know that  $\Sigma$  is diagonalizable, since it is symmetric. If  $\Sigma$  is  $k \times k$ , and the respective eigenvectors and eigenvalues are  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$  and  $\lambda_1, \lambda_2, \dots, \lambda_k$ . Then the total variance is  $\lambda_1 + \lambda_2 + \dots + \lambda_k$ , and the fraction of variance explained by eigenvector  $\mathbf{v}_i$  is

$$\frac{\lambda_i}{\lambda_1 + \lambda_2 + \dots + \lambda_i + \dots + \lambda_k}.$$

# Eigenvectors and Eigenvalues Example

## Example

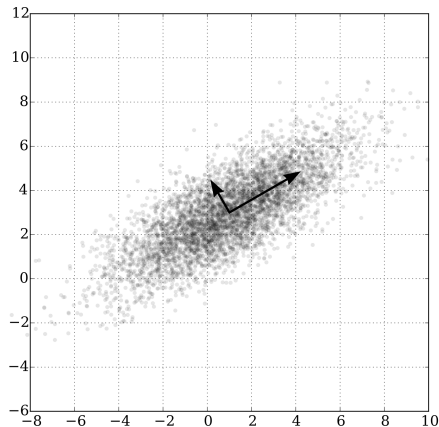
Consider  $\Sigma = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$ . Find the eigenvectors and eigenvalues.

# Principal Component Analysis Definition

## Definition

**Principal Component Analysis (PCA)** is a method for identifying the optimal orthogonal projection of a dataset into a lower-dimensional subspace. Among all possible linear transformations, PCA is the best representation of the data in the sense that it maximizes the captured variance for any chosen dimension  $\ell$ . The **principal components** are the  $\ell$  eigenvectors corresponding to the largest  $\ell$  eigenvalues. These vectors define the directions that minimize the total squared projection distance between the original data points and the new subspace, or equivalently, maximize the explained variance.

# Principal Component Analysis Figure



# Principal Component Analysis Steps

1. **Collect and Clean:** Prepare your data matrix  $X$  of dimension  $n \times k$ . Perform any necessary data cleaning.
2. **Center and Scale:** Demean your data  $\tilde{x} = x - \mu$  to obtain  $\tilde{X}$ . Standardize, i.e. use  $\tilde{x} = (x - \mu)/\sigma$ , if features have different scales.
3. **Decomposition:** Compute the eigenvalues  $\lambda_i$  and the corresponding eigenvectors  $\mathbf{v}_i$  of the covariance matrix  $\Sigma$ .
4. **Selection:** Determine the number  $\ell$  of components to use. This is usually either driven by your application or via a desire to explain a particular fraction of the variance, e.g. 95% of the variance. We want to reduce the dimension so  $\ell < k$ .
5. **Projection:** Select the  $\ell$  principal components (i.e. eigenvectors) corresponding to the largest  $\ell$  eigenvalues to form the optimal orthogonal basis  $V_\ell$ .
6. **Transform:** Project  $\tilde{X}$  onto the new subspace. The  $n \times \ell$  matrix  $\tilde{X}V_\ell$  contains the loadings of the projection.
7. **Interpretation:** Analyze the loadings, i.e. the coefficients in front of each eigenvector in the orthonormal basis  $V_\ell$ .

# Orthonormal Basis Definition

## Definition

An **orthonormal basis** of a vector space  $V$  of dimension  $k$  is a basis  $(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k)$  such that

- $\mathbf{v}_i \bullet \mathbf{v}_j = 0$  for  $i \neq j$  and
- $\|\mathbf{v}_i\| = 1$  for  $i = 1, 2, \dots, k$ .

# Orthonormal Basis

Suppose  $\Sigma$  is the covariance matrix corresponding to the  $n \times k$  matrix  $X$ . The eigenvectors of  $\Sigma$  form the orthonormal eigenbasis  $(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k)$ . So, for any  $\mathbf{u}$  in  $\mathbb{R}^k$ , we can write

$$\mathbf{u} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_\ell \mathbf{v}_\ell + \dots + \alpha_k \mathbf{v}_k$$

for unique  $\alpha_i$

# What are Loadings?

Because the basis  $(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k)$  is orthonormal, we know

$$\mathbf{u} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_\ell \mathbf{v}_\ell + \dots + \alpha_k \mathbf{v}_k$$

implies the **loading**

$$\alpha_i = \frac{\mathbf{u} \bullet \mathbf{v}_i}{\|\mathbf{v}_i\|^2} = \mathbf{u} \bullet \mathbf{v}_i.$$

# The First $\ell$ Principal Components

If we have

$$\mathbf{u} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_\ell \mathbf{v}_\ell + \dots + \alpha_k \mathbf{v}_k$$

then the best way to reduce linearly the dimension while preserving as much of the explained variance as possible is to assume

$$\mathbf{u} \approx \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_\ell \mathbf{v}_\ell.$$

# Principal Component Analysis Example

## Example

Generate 100 points from  $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ , where

$$\boldsymbol{\mu} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \text{and} \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}.$$

Then plot lines in the directions of the first two principal components.

# Principal Component Analysis Example Solution

```
# Import modules
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

# Use Seaborn style
plt.style.use('seaborn-v0_8')

# Set random seed
np.random.seed(0)

# Get covariance matrix
cov = np.array([[1, 0.5], [0.5, 1]])

# Generate data
X = multivariate_normal.rvs(
    mean = np.zeros(2),
    cov = cov,
    size = 100)

# Demean data, though mean already close to
# zero
X -= X.mean(axis = 0)

# Calculate covariance matrix
S = np.cov(X, rowvar = False, ddof = 1)

# Get eigenvalues and eigenvectors
evals, evecs = np.linalg.eigh(S)

# Index to sort eigenvalues into descending
# order
idx = evals.argsort()[::-1]
```

```
# Sort eigenvalues and -vectors
evals, evecs = evals[idx], evecs[:, idx]

# Get slopes
m_1 = evecs[1, 0]/evecs[0, 0]
m_2 = evecs[1, 1]/evecs[0, 1]

# Get x-values for plot
x_vals = np.linspace(X.min(), X.max())

# Plot graph
plt.scatter(*X.T, label = 'Data')
plt.plot(x_vals, m_1 * x_vals,
         color = 'red', label = 'PC1 Line')
plt.plot(x_vals, m_2 * x_vals,
         color = 'orange', label = 'PC2 Line')

# Add legend
plt.legend()

# Make aspect ratio the same, so PCs look
# perp
plt.axis('equal')

# Add title
plt.title('Data and Principal Component
          Lines')

# Save the figure
plt.savefig(path + r'ex4-2.png')

plt.show()
```

# Principal Component Analysis Example Result



# Sample Covariance Matrix Example

Here is some code to get the sample covariance matrix for the *current* S&P constituents. The uploaded data are the daily constituents' returns from January 1, 2021 to May 31, 2026 which are available over the whole time range. The data source is *Yahoo! Finance*. Check out the Unit 4 Code Snippets to see how the data were extracted.

```
# Import modules
import pandas as pd

# Load in data; make date the index
data = pd.read_csv(data_path, index_col = 'Date')

# Get covariance matrix
S = data.cov()

S
```

# Sample Covariance Matrix Result

The output looks like this:

| Ticker<br>Date | A        | AAPL      | ABBV      | ABNB      | ABT       | ACGL      | ACN       | ADBE      | ADI       | ADM       | ... | WY        | WYNN      | XEL       | XOM      | X       |
|----------------|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----|-----------|-----------|-----------|----------|---------|
| 2021-01-04     | 0.002908 | -0.024719 | -0.016239 | -0.052112 | -0.003471 | -0.032437 | -0.018185 | -0.029553 | -0.011575 | -0.010712 | ... | -0.021473 | -0.052557 | -0.015149 | 0.006793 | -0.0206 |
| 2021-01-05     | 0.008176 | 0.012363  | 0.010341  | 0.065756  | 0.012373  | 0.004011  | 0.005693  | 0.000721  | 0.017874  | 0.019450  | ... | 0.009448  | 0.030776  | -0.009747 | 0.048193 | -0.0012 |
| 2021-01-06     | 0.027423 | -0.033661 | -0.008638 | -0.037289 | -0.002082 | 0.043950  | 0.010933  | -0.039902 | 0.004508  | 0.040519  | ... | 0.005133  | 0.005990  | 0.022147  | 0.025517 | 0.0421  |
| 2021-01-07     | 0.026609 | 0.034123  | 0.010703  | 0.059536  | 0.009707  | -0.009295 | 0.009435  | 0.024512  | 0.042264  | -0.008885 | ... | 0.009613  | -0.009923 | -0.019560 | 0.007846 | 0.0453  |
| 2021-01-08     | 0.007134 | 0.008631  | 0.005248  | -0.009916 | 0.002785  | 0.005519  | 0.003648  | 0.015406  | 0.007262  | 0.002670  | ... | -0.006843 | -0.006560 | 0.009208  | 0.011121 | -0.0101 |

5 rows × 490 columns

Note: There are fewer than 500 columns because some of the current S&P constituents were not publicly traded companies in 2021.

# Principal Component Analysis Example

## Example

Use PCA to represent the S&P 500 constituent data from before on a two dimensional scatter plot.

# Principal Component Analysis Example Solution

```
# Import modules
import numpy as np
import matplotlib.pyplot as plt

# Increase resolution
plt.rcParams['figure.dpi'] = 300

# Use Seaborn style
plt.style.use('seaborn-v0.8')

# Get S&P 500 return cov matrix
S = data.cov()

# Get the eigenvalues and eigenvectors
evals, evects = np.linalg.eigh(S)

# Index to sort eigenvalues into descending
# order
idx = evals.argsort()[::-1]

# Change order
evals, evects = evals[idx], evects[:, idx]

# Convert the observations to a numpy array
X = data.values

# Demean X
X -= X.mean(axis = 0)

# Calculate loadings using the dot product
loadings = X @ evects[:, 0:2]

# Unpack results
x, y = zip(*loadings)

# Get scatter plot
plt.scatter(x, y)

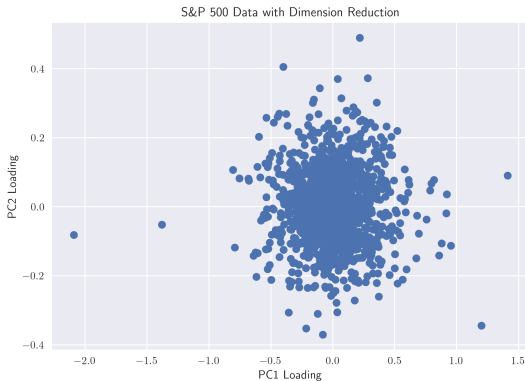
# Create x- and y-labels
plt.xlabel('PC1 Loading')
plt.ylabel('PC2 Loading')

# Give the plot a title
plt.title(r'S&P 500 Data with Dimension
          Reduction')

# Save the figure
plt.savefig(path + r'ex4-3.png')

plt.show()
```

# Principal Component Analysis Example Result



# Reconstruction

If we have data with mean  $\boldsymbol{\mu}$ , and we used the loadings of the first  $\ell$  eigenvectors  $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_\ell$  to approximate the vector  $\mathbf{x}$ , then this is making the assumption

$$\mathbf{x} \approx \boldsymbol{\mu} + \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_\ell \mathbf{v}_\ell,$$

where  $\alpha_j$  is the loading corresponding to  $\mathbf{v}_j$ .

# Why Standardize?

PCA is sensitive to the units of measurement. If one variable has a much larger variance simply due to its scale, PCA will treat it as a “Principal Component” regardless of its actual importance.

- Standardize when variables have different units (e.g., Yen vs. USD).
- Demean only when variables are in the same units and relative volatility is a signal you want to preserve (e.g., across the yield curve).

# Clipping Covariance Matrices

# Positive Definite Matrices

Using the dot product, a matrix  $S$  is *positive definite* if

$$\mathbf{x}^T S \mathbf{x} > 0 \quad \text{for all} \quad \mathbf{x} \neq \mathbf{0}.$$

Since a covariance matrix is diagonalizable due to the spectral theorem, a covariance matrix will be positive definite if and only if all its eigenvalues are positive.

# Non-Positive Eigenvalues

- **Numerical Errors:** Negative eigenvalues do not make sense and are due to floating-point round-off error.
- **Linear Dependency:** A zero eigenvalue implies the covariance matrix is non-invertible. This often occurs when the dimension is larger than the number of observations, i.e.  $n < k$ .
- **Invertibility:** Many financial applications require the inverse of the covariance matrix to exist. If any eigenvalue is zero, the matrix is non-invertible. Even eigenvalues simply close to zero can cause issues in applications.

# Non-Positive Eigenvalues Python Example

## Example

Consider a data matrix  $X$  of size  $n \times k$ , where  $n = 50$  and  $k = 100$ . Suppose the rows of  $X$  are drawn from a multivariate normal distribution  $\mathcal{N}(0, I_k)$ , where  $I_k$  is the  $k \times k$  identity matrix. Use Python to generate an  $X$  that matches this description and calculate the sample covariance matrix.

# Non-Positive Eigenvalues Python Example Solution

```
# Import modules
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal

# Set the random seed
np.random.seed(0)

# Define the dimensions
N, K = 50, 100

# Generate normal random variables
X = multivariate_normal.rvs(mean = np.zeros(K),
                             cov = np.eye(K),
                             size = N)

# Get the covariance matrix
S = np.cov(X, rowvar = False, ddof = 1)

# Get the eigenvalues
evals, _ = np.linalg.eigh(S)

print(f'The sample covariance matrix has {np.isclose(evals, 0).sum()} ',
      r'eigenvalues numerically indistinguishable from 0.')
```

It says there are 51 eigenvalues numerically indistinguishable from 0. The true covariance matrix is the  $100 \times 100$  identity and therefore the true eigenvalues are all equal to 1.

# Clip the Covariance Matrix

One solution is to “clip” the sample covariance matrix. That is, simply replace all the eigenvalues that are too small with something bigger and reconstruct the covariance matrix.

# Clip the Covariance Matrix Example

## Example

The previous sample covariance matrix  $S$  is singular because  $n < k$ . To ensure  $S$  is positive definite (and therefore invertible), apply a lower truncation of  $10^{-7}$  to all eigenvalues of the corresponding correlation matrix  $C$ , then reconstruct the covariance matrix  $S_{\text{new}}$ .

# Clip the Covariance Matrix Example

```
# Get the standard deviations
stds = np.sqrt(np.diag(S))

# Get the correlation matrix
C = np.diag(1/stds) @ S @ np.diag(1/stds)

# Get the eigenvalues and vectors of C
evals_c, evecs_c = np.linalg.eigh(C)

# Make lam_min small positive outside of np
.isclose threshold
lam_min = 1e-7

# Replace eigenvalues that are too small
evals_c[evals_c < lam_min] = lam_min

# Reconstruct correlation matrix
C_new = evecs_c @ np.diag(evals_c) @
        evecs_c.T

# Make sure still correlation matrix
C_new = (np.diag(np.sqrt(1/np.diag(C_new)))
        @ C_new
        @ np.diag(np.sqrt(1/np.diag(
        C_new))))

# Multiply by standard deviations to make
it covariance matrix
S_new = np.diag(stds) @ C_new @ np.diag(
        stds)

# Get the eigenvalues
evals_new, _ = np.linalg.eigh(S_new)

print(f'The new covariance matrix has {np.
        isclose(evals_new, 0).sum()} ',
        r'eigenvalues numerically
        indistinguishable from 0.')
```

Now it says all the eigenvalues are positive! Inspection shows that the covariance matrix estimate is otherwise very similar to the sample covariance matrix.

# Marchenko–Pastur Theorem

## Theorem

Consider a matrix of independent and identically distributed random observations  $X$  of size  $n \times k$ , where the underlying process generating the observations has mean 0 and variance  $\sigma^2$ . If  $q = n/k \geq 1$  is constant, the covariance matrix  $S = \frac{1}{n}X^T X$  has eigenvalues that converges to a distribution with probability density function

$$f(\lambda) = \begin{cases} \frac{q}{2\pi\sigma^2} \frac{\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}}{\lambda}, & \lambda_- \leq \lambda \leq \lambda_+ \\ 0, & \text{otherwise} \end{cases}$$

where

$$\lambda_- = \sigma^2 \left(1 - \sqrt{\frac{1}{q}}\right)^2 \quad \text{and} \quad \lambda_+ = \sigma^2 \left(1 + \sqrt{\frac{1}{q}}\right)^2.$$

If  $q < 1$ , the matrix  $S$  has  $k - n$  zero eigenvalues. The Marchenko–Pastur density remains valid for  $\lambda > 0$ , but the distribution gains a point mass of weight  $1 - q$  at  $\lambda = 0$ .

# Marchenko–Pastur Python Example

## Example

Consider a  $50,000 \times 10,000$  matrix  $X$  containing independent and identically distributed standard normal entries. Let  $C$  be the correlation matrix calculated from  $X$ . Generate  $X$  and compute the empirical distribution of the eigenvalues of  $C$ , and compare it to the theoretical Marchenk–Pastur distribution.

# Marchenko–Pastur Python Example Solution

```
# Import modules
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Use LaTeX and increase resolution
plt.rcParams['text.usetex'], plt.rcParams['figure.dpi'] = True, 300

# Use Seaborn style
plt.style.use('seaborn-v0.8')

# Set the random seed
np.random.seed(0)

# Start the clock
start_time = time.perf_counter()

# Generate multivariate normal random variables
X = norm.rvs(size = (50_000, 10_000))
```

# Marchenko–Pastur Python Example Solution

```
# Get the number of observations and variables
N, K = X.shape

# Get correlation matrix
C = np.corrcoef(X, rowvar = False)

# Clear up RAM
del X

# q is the number of observations divided by the number of variables
q = N/K

# Get eigenvalues
evals, _ = np.linalg.eigh(C)

# Get support of Marchenko—Pastur distribution
lam_minus, lam_plus = (1 - np.sqrt(1/q))**2, (1 + np.sqrt(1/q))**2

# Define pdf
def f(lam):
    # Support of Marchenko—Pastur distribution
    in_support = ((lam_minus <= lam) & (lam <= lam_plus)).astype(float)
    return q/(2 * np.pi) * np.sqrt((lam_plus - lam) * (lam - lam_minus))/lam * in_support
```

# Marchenko–Pastur Python Example Solution

```
# Get lam_vals
lam_vals = np.linspace(lam_minus, lam_plus, 100)

# Get density values
f_vals = f(lam_vals)

# Plot histogram
plt.hist(evals, density = True, bins = int(np.sqrt(K)), label = 'Simulated Distribution')

# Plot density
plt.plot(lam_vals, f_vals, label = 'Density')

# Add legend
plt.legend()

# Add x-label
plt.xlabel(r'$\lambda$')

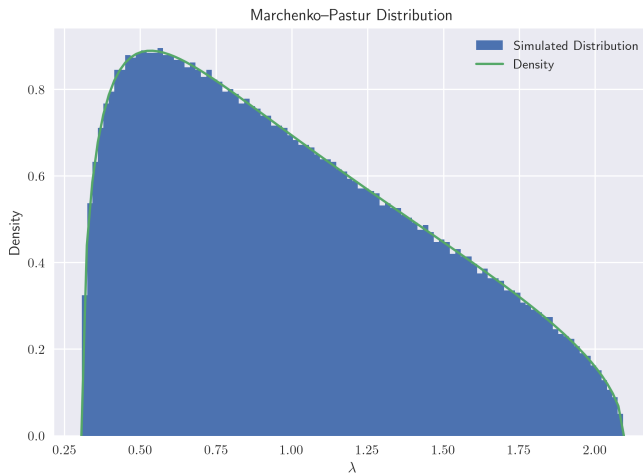
# Add y-label
plt.ylabel('Density')

# Add title to plot
plt.title('Marchenko–Pastur Distribution')

# Save the figure
plt.savefig(path + r'ex4-4.png')

plt.show()
```

# Marchenko–Pastur Python Example Result



# How to use it?

Consider eigenvalue  $\lambda$  of the covariance matrix.

- If  $\lambda > \lambda_+$ , then the result is consistent with signal.
- If  $\lambda \leq \lambda_+$ , the result is probably just random noise. Replace values less than  $\lambda_+$  with mean of values less than  $\lambda_+$  to make the covariance matrix more stable.

# S&P 500 Constituent Example

## Example

Use the S&P 500 Constituent sample covariance matrix, and plot the eigenvalues. Compare the results to the corresponding Marchenko–Pastur distribution.

# S&P 500 Constituent Example Solution

Suppose  $S$  is the sample covariance matrix which we calculated from the S&P 500 constituent data before.

```
# Get the standard deviations
stds = np.sqrt(np.diag(S))

# Calculate correlation matrix
C = np.diag(1/stds) @ S @ np.diag(1/stds)

# Get the eigenvalues and vectors
evals, evecs = np.linalg.eigh(C)

# Save q
q = data.shape[0]/data.shape[1]

# Get support of Marchenko—Pastur distribution
lam_minus, lam_plus = (1 - np.sqrt(1/q))**2, (1 + np.sqrt(1/q))**2

# Define pdf
def f(lam):
    # Support of Marchenko—Pastur distribution
    in_support = ((lam_minus <= lam) & (lam <= lam_plus)).astype(float)
    return q/(2 * np.pi) * np.sqrt((lam_plus - lam) * (lam - lam_minus))/lam * in_support

# Get lam_vals
lam_vals = np.linspace(lam_minus, lam_plus, 100)

# Get density values
f_vals = f(lam_vals)
```

# S&P 500 Constituent Example Solution

```
# Plot eigenvalues that are signal
plt.scatter(evals[evals > lam_plus], np.zeros((evals > lam_plus).sum()),
            label = 'Signal Eigenvalues')

# Plot eigenvalues that are noise
plt.scatter(evals[evals <= lam_plus], np.zeros((evals <= lam_plus).sum()),
            label = 'Noise Eigenvalues', color = 'gray')

# Plot density
plt.plot(lam_vals, f_vals, label = 'Density', color = 'green')

# There are 3 eigenvalues much larger than 10
plt.xlim([0, 10])

# Add legend
plt.legend()

# Add x-label
plt.xlabel(r'$\lambda$')

# Add y-label
plt.ylabel(r'Density')

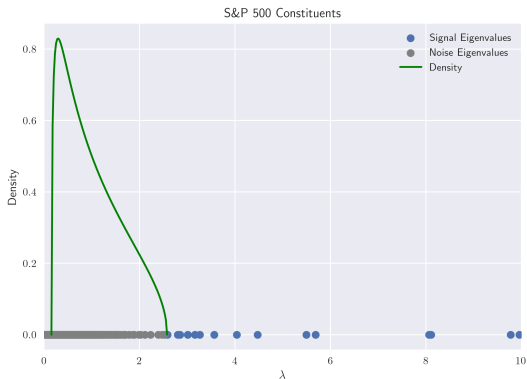
# Add title to plot
plt.title(r'S&P 500 Constituents')

# Save the figure
plt.savefig(path + r'ex4-5.png')

plt.show()
```

# S&P 500 Constituent Example Result

Only the values past  $\lambda_+$  are consistent with signal. There are three eigenvalues greater than 10 which are not shown.



# Clip the S&P 500 Constituent Covariance Matrix Example

## Example

Clip the S&P 500 constituent covariance matrix. Replace all values less than the upper value of the Marchenko–Pastur support with the mean value within the support.

# Clip the S&P 500 Constituent Covariance Matrix Example Solution

```
# Initialize new eigenvalues
evals_new = evals.copy()

# Replace noise eigenvalues with mean
evals_new[evals_new < lam_plus] = evals_new[
    (evals_new < lam_plus) & (evals_new > 0)].mean()

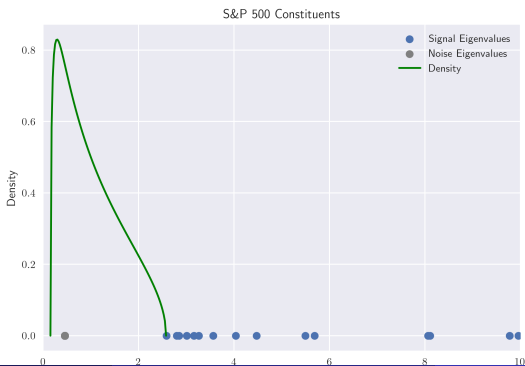
# Construct new correlation matrix
C_new = evecs @ np.diag(evals_new) @ evecs.T

# Make sure still correlation matrix
C_new = np.diag(1/np.sqrt(np.diag(C_new))) @ C_new @ np.diag(1/np.sqrt(np.diag(C_new)))

# Make new covariance matrix
S_new = np.diag(stds) @ C_new @ np.diag(stds)
```

# Clip the S&P 500 Constituent Covariance Matrix Example Result

The signal eigenvalues are the same, while the noise eigenvalues have been replaced with the mean.



# Stochastic Calculus

I'm following these notes very closely:

<http://www.columbia.edu/~mh2078/FoundationsFE/IntroStochCalc.pdf>

# Probability Triple

We assume we have the probability space  $(\Omega, \mathcal{F}, P)$  where

- $\Omega$  is the universe of possible outcomes,
- $\mathcal{F}$  represents the  $\sigma$ -algebra of events in  $\Omega$ , and
- $P$  is the “true” or physical probability measure.

# Filtration Definition

## Definition

A **filtration**  $\{\mathcal{F}_t\}_{t \geq 0}$  is a set of  $\sigma$ -algebras that models the evolution of information through time. Since information increases over time  $\mathcal{F}_s \subseteq \mathcal{F}_t$  for  $s < t$ . If it is known by time  $t$  whether or not an event  $E$  has occurred, then we have  $E \in \mathcal{F}_t$ . If we are working with a finite horizon  $[0, T]$ , then we can take  $\mathcal{F} = \mathcal{F}_T$ .

# Filtration Example

Consider a stock that can move up with probability  $p$  or down with probability  $1 - p$  each period. Suppose stock moves are independent.

- $t = 0$  : No information has been revealed, so  $\mathcal{F}_0 = \{\emptyset, \Omega\}$ . The probability  $P(\text{up-up}|\mathcal{F}_0) = p^2$ .
- $t = 1$  : The first move is observed. If we saw up, then the event up is in  $\mathcal{F}_1$ . Inside  $\mathcal{F}_1$ , the probability  $P(\text{up}|\mathcal{F}_1) = 1$ . The probability  $P(\text{up-up}|\mathcal{F}_1) = p$ .
- $t = 2$  : Both moves are revealed. Every possible path (up-up, up-down, down-up, down-down) is now an element of  $\mathcal{F}_2$ . If, say, we observed up-down, then  $P(\text{up-down}|\mathcal{F}_2) = 1$  and  $P(\text{up-up}|\mathcal{F}_2) = 0$ .

# Stochastic Process Definition

## Definition

For a given probability space  $(\Omega, \mathcal{F}, P)$ , a **stochastic process** is a collection of random variables indexed by  $\mathcal{T}$ . We write  $\{X_t \mid t \in \mathcal{T}\}$  to denote a stochastic process, and we think of  $\mathcal{T}$  as the time index. Sometimes we simply write  $X_t$  and only implicitly refer to the index.

# Stochastic Process Example

## Example

For a high yield bond portfolio, we can model the total number of defaulted bonds this year up to day  $t$  as a stochastic process. Denote the number of defaulted bonds on day  $t$  by  $N_t$ . In this case,  $\mathcal{T} = \{1, 2, 3, \dots, 365\}$ , assuming there are 365 days in a year. Using our prior notation, the stochastic process is  $\{N_t \mid t \in \mathcal{T}\}$ .

# $\mathcal{F}_t$ -Adapted Definition

## Definition

We say that a stochastic process  $X_t$  is  $\mathcal{F}_t$ -**adapted** if for every  $t$  in  $\mathcal{T}$  the information about  $X_t$  is contained in  $\mathcal{F}_t$ .

# Brownian Motion Definition

## Definition

A stochastic process  $\{W_t \mid t \geq 0\}$  is a **standard Brownian motion** if the following hold.

BM1  $W_0 = 0$ .

BM2 It has continuous sample paths.

BM3  $W_{t_1} - W_{s_1}$  and  $W_{t_2} - W_{s_2}$  are independent for  $0 \leq s_1 \leq t_1 \leq s_2 \leq t_2$ .

BM4  $W_t - W_s \sim \mathcal{N}(0, t - s)$  for  $0 \leq s \leq t$ .

# Simulating Brownian Motion

Suppose that we want to simulate a Brownian motion on the interval  $[0, T]$ . Then construct a partition of the interval

$$0 = t_0 < t_1 < \dots < t_{n-1} < t_n = T.$$

For  $i = 1, 2, \dots, n$ , generate  $Z_i \sim \mathcal{N}(0, 1^2)$ . Then

$$\widetilde{W}_{t_k} = \begin{cases} 0, & k = 0 \\ \sum_{i=1}^k Z_i \sqrt{\Delta t_i}, & k = 1, 2, \dots, n \end{cases}$$

is “approximately” Brownian motion. These simulated values are exactly Brownian motions at  $t_0, t_1, t_2, \dots$ , and  $t_n$ .

# Simulating Brownian Motion Example

## Example

Use Python to simulate five Brownian motions on the interval  $[0, 1]$ .

# Simulating Brownian Motion Example Solution

```
# Import modules
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Use LaTeX and increase resolution
plt.rcParams['text.usetex'], plt.rcParams['figure.dpi'] = True, 300

# Use Seaborn style
plt.style.use('seaborn-v0.8')

# Set the random seed
np.random.seed(0)

# Break up so exact BM at  $n + 1$  times
n = 500

# Simulate five Brownian motions
for _ in range(5):
    # Simulate Brownian motion for  $t$  in  $[0, 1]$ 
    dW = norm.rvs(scale = np.sqrt(1/n), size = n)

    # Take the cumulative sum; add 0 for the  $t = 0$  value
    W = np.insert(np.cumsum(dW), 0, 0)

    # Plot results
    plt.plot(np.linspace(0, 1, n + 1), W)

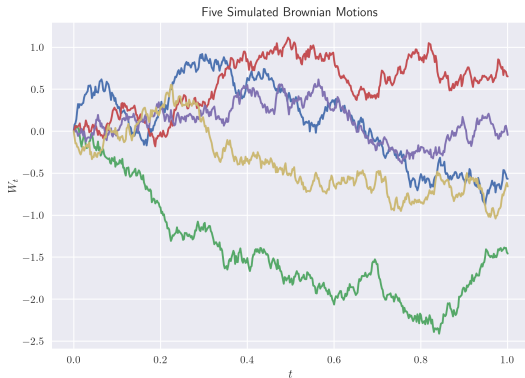
# Add x- and y-label as well as title
plt.xlabel(r'$t$'); plt.ylabel(r'$W_t$')
plt.title(r'Five Simulated Brownian Motions')

# Save the figure
plt.savefig(path + r'ex4-7.png')

plt.show()
```

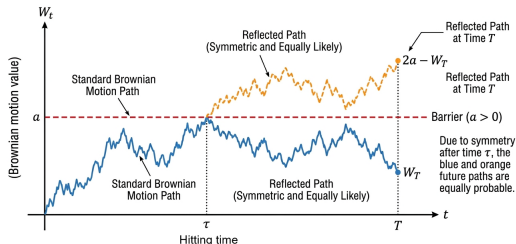
# Simulating Brownian Motion Example Result

Five Brownian motions on the interval  $[0, 1]$ , using a uniform partition that breaks  $[0, 1]$  into 500 subintervals.



# The Reflection Principle

Let  $W_t$  be a standard Brownian motion and  $t = \tau$  be the first hitting time of a barrier  $a > 0$ , i.e.  $W_\tau = a$  and  $W_t \neq a$  for  $t < \tau$ . Given  $\tau \leq T$ , the subsequent path and the path reflected across the horizontal line  $y = a$  are probabilistically indistinguishable.



# The Reflection Principle Example

## Example

Calculate the probability that a standard Brownian motion crosses a barrier of  $a = 2$  before time  $T = 1$ .

# Martingale Definition

## Definition

A stochastic process  $\{X_t \mid t \geq 0\}$  is a **martingale** with respect to the filtration  $\mathcal{F}_t$  if the following hold.

M1  $E[|X_t|] < \infty$  for all  $t \geq 0$ .

M2  $E[X_{t+s} | \mathcal{F}_t] = X_t$  for all  $t, s \geq 0$ .

# Martingale Example

## Example

Prove the following are martingales.

(a)  $W_t$

(b)  $W_t^2 - t$

(c)  $\exp\left(\theta W_t - \frac{\theta^2 t}{2}\right)$

# Quadratic Variation Definition

## Definition

Let  $X_t$  be some stochastic process. The **quadratic variation** of a stochastic process  $X_t$  is

$$\lim_{\|\mathcal{P}\| \rightarrow 0} \sum_{k=1}^n \left( X_{t_k} - X_{t_{k-1}} \right)^2,$$

where  $\mathcal{P}$  is an arbitrary partition of  $[0, T]$  and  $\|\mathcal{P}\| = \max_k \{\Delta t_k\}$  is the mesh of the partition.

# Quadratic Variation Example

## Example

Compute the quadratic variation of the deterministic process  $X_t = t^2$  over the interval  $[0, T]$ .

# Quadratic Variation of Differentiable Functions

Any differentiable function  $f$  will end up having quadratic variation 0, like we saw for  $t^2$  in our example. To see why, we note

$$\frac{f(t_k) - f(t_{k-1})}{t_k - t_{k-1}} \rightarrow f' \left( \frac{t_k + t_{k-1}}{2} \right) \quad \text{as } \|\mathcal{P}\| \rightarrow 0.$$

Therefore,

$$\begin{aligned} \lim_{\|\mathcal{P}\| \rightarrow 0} \sum_{k=1}^n \left( f(t_k) - f(t_{k-1}) \right)^2 &= \lim_{\|\mathcal{P}\| \rightarrow 0} \sum_{k=1}^n \left( f' \left( \frac{t_k + t_{k-1}}{2} \right) \right)^2 (t_k - t_{k-1})^2 \\ &\leq \lim_{\|\mathcal{P}\| \rightarrow 0} \|\mathcal{P}\| \sum_{k=1}^n \left( f' \left( \frac{t_k + t_{k-1}}{2} \right) \right)^2 \Delta t_k \\ &= 0. \end{aligned}$$

# Quadratic Variation of a Brownian Motion

## Theorem

*The quadratic variation of a standard Brownian motion  $\{W_t \mid 0 \leq t \leq T\}$  is equal to  $T$  with probability 1.*

## Theorem (Levey's Theorem)

*A continuous martingale is a standard Brownian motion if and only if its quadratic variation over each interval  $[0, T]$  is equal to  $T$ .*

# Approximate Quadratic Variation Example

## Example

Approximate Brownian motion in the interval  $[0, 1]$  with finer and finer partitions. Observe that as the partitions get finer, the quadratic variation approaches a straight line.

# Approximate Quadratic Variation Example Solution

```
# Import modules
import numpy as np, matplotlib.pyplot as plt
from scipy.stats import norm

# Use LaTeX and increase resolution
plt.rcParams['text.usetex'], plt.rcParams['figure.dpi'] = True, 300

# Use Seaborn style
plt.style.use('seaborn-v0_8')

# Set the random seed
np.random.seed(0)

# Define n-values
n_vals = [10, 100, 1000, 10000]

# Set up subplots
fig, ax = plt.subplots(2, 2, sharey = True, figsize = (15, 10))

# Simulate five Brownian motions
for i, n in enumerate(n_vals):
    # Get row and column
    row, col = i//2, i % 2

    # Difference of two Brownian motions is normal
    dW = norm.rvs(scale = np.sqrt(1/n), size = (3, n))

    # Calculate quadratic variance
    quad_var = np.cumsum(dW**2, axis = 1)
```

# Approximate Quadratic Variation Example Solution

```
# Plot results
for j in range(3):
    ax[row, col].plot(np.linspace(1/n, 1, n), quad_var[j, :], label = f'Brownian
    Motion {j}')
# Plot t
ax[row, col].plot(np.linspace(1/n, 1, n), np.linspace(1/n, 1, n), label = r'$t$')
# Add x-label
ax[row, col].set_xlabel(r'$t$')
# Add y-label
ax[row, col].set_ylabel(r'Approximate Quadratic Variation')
# Give plot a title
ax[row, col].set_title(f'$n$ = {n}')
# Add legend
ax[row, col].legend()

# Add title to plot
plt.suptitle(r'Approximate Quadratic Variation  $\sum_{k=1}^n (W_{t_k} - W_{t_{k-1}})^2$ ')

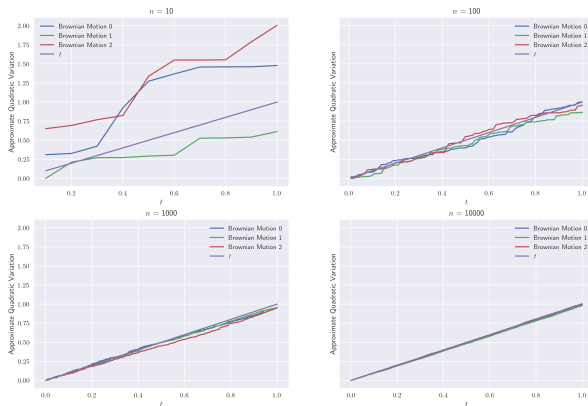
# Save the figure
plt.savefig(path + r'ex4-8.png')

plt.show()
```

# Approximate Quadratic Variation Example Result

From the four subplots, we see that  $\lim_{n \rightarrow \infty} \sum_{k=1}^n (W_{t_k} - W_{t_{k-1}})^2 = T$ .

$$\text{Approximate Quadratic Variation} = \sum_{k=1}^n (W_{t_k} - W_{t_{k-1}})^2$$



# Itô Integral Definition

## Definition

The **Itô Integral** of  $X_t$  with respect to standard Brownian motion

$$\int_0^T X_t dW_t = \lim_{\|P\| \rightarrow 0} \sum_{k=1}^n X_{t_{k-1}} (W_{t_k} - W_{t_{k-1}}),$$

where  $\mathcal{P} = (t_0, t_1, \dots, t_n)$  is an arbitrary partition of  $[0, T]$ .

# Itô Integral Example

## Example

Assuming the Itô integral exists, compute  $\int_0^T W_t dW_t$ .

# Itô Integral Python Example

## Example

Check the previous result by running a simulation in Python.

# Itô Integral Python Example

```
# Import modules
import numpy as np
from scipy.stats import norm

# Set the random seed
np.random.seed(0)

# Set n, so there are n + 1 places where BM exact
n = 1_000_000

# Define T
T = 1

# Simulate independent increment
dW = norm.rvs(scale = np.sqrt(T/n), size = n)

# Sum up to get BM
W = np.insert(np.cumsum(dW), 0, 0)

# Approximate Ito integral; using left endpoints
integral_simulated = np.sum(W[0:-1] * dW)

# Write analytic result
integral_analytic = 1/2 * W[-1]**2 - T/2

print(f'The simulated result is {integral_simulated:.3f} while the analytic result is {
    integral_analytic:.3f}.')
```

The output reads:

*The simulated result is 0.643 while the analytic result is 0.643.*

# Law of Iterated Expectations

Suppose that  $s \leq t \leq T$ . Then

$$E[X_T | \mathcal{F}_s] = E\left[E[X_T | \mathcal{F}_t] \middle| \mathcal{F}_s\right].$$

Instead of writing  $E[X_T | \mathcal{F}_t]$  we often write  $E_t[X_T]$ . Using this notation, the Law of Iterated Expectations is

$$E_s[X_T] = E_s\left[E_t[X_T]\right].$$

# Itô Isometry

## Theorem (Itô Isometry)

*We have*

$$E \left[ \left( \int_0^T X_t dW_t \right)^2 \right] = E \left[ \int_0^T X_t^2 dt \right]$$

*whenever*

$$E \left[ \int_0^T X_t^2 dt \right] < \infty.$$

# Itô Integral Example

## Example

Prove that the stochastic process

$$Y_t = \int_0^t X_u dW_u$$

is a martingale.

# Stochastic Differential Equation Definition

## Definition

We define the **stochastic differential equation**

$$dX_t = a(t, X_t) dt + b(t, X_t) dW_t$$

to mean

$$X_t = X_0 + \int_0^t a(s, X_s) ds + \int_0^t b(s, X_s) dW_s.$$

These rules hold for stochastic differential equations

$$dt^2 = 0, \quad dt \, dW_t = 0, \quad \text{and} \quad dW_t^2 = dt$$

# Box Calculus Example

## Example

Suppose

$$dS_t = \mu S_t dt + \sigma S_t dW_t.$$

Use box Calculus to compute  $dS_t^2$ .

# Itô's Lemma

## Theorem

*Suppose that  $dX_t = \mu_t dt + \sigma_t dW_t$ , and  $Z_t = f(t, X_t)$ , where  $f(t, x)$  is continuously differentiable at least once with respect to  $t$  and at least twice with respect to  $x$ . Then*

$$dZ_t = \left( \frac{\partial f}{\partial t}(t, X_t) + \mu_t \frac{\partial f}{\partial x}(t, X_t) + \frac{\sigma_t^2}{2} \frac{\partial^2 f}{\partial x^2}(t, X_t) \right) dt + \sigma_t \frac{\partial f}{\partial x}(t, X_t) dW_t.$$

# Itô's Lemma Example 1

## Example

Suppose  $dS_t = \mu S_t dt + \sigma S_t dW_t$ . Use Itô's Lemma to find  $d(\log S_t)$ .

# Itô's Lemma Example 2

## Example

Use Itô's Lemma or box calculus to find another way to compute

$$\int_0^T W_t dW_t.$$